

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 23)

“Debugging FreeRTOS
Applications”



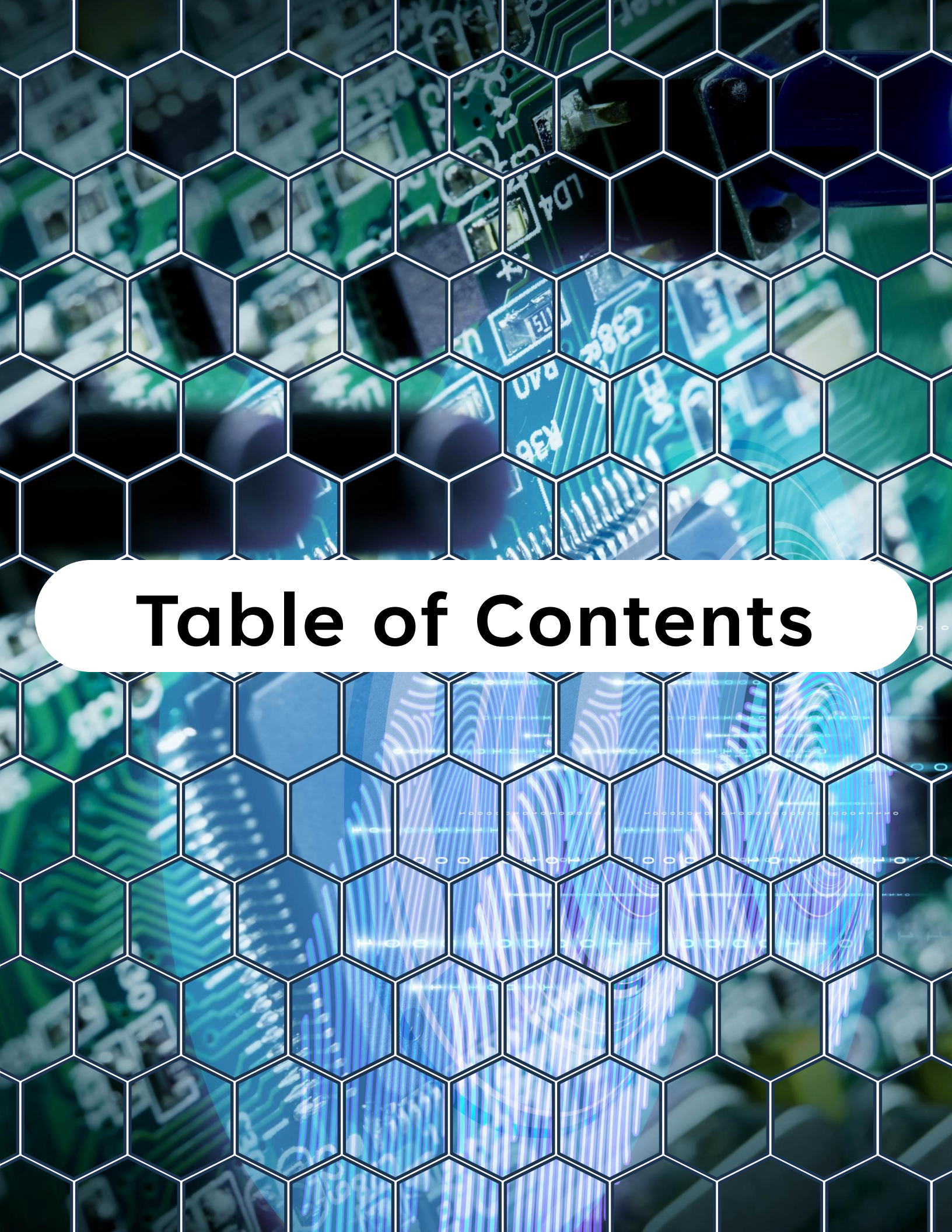
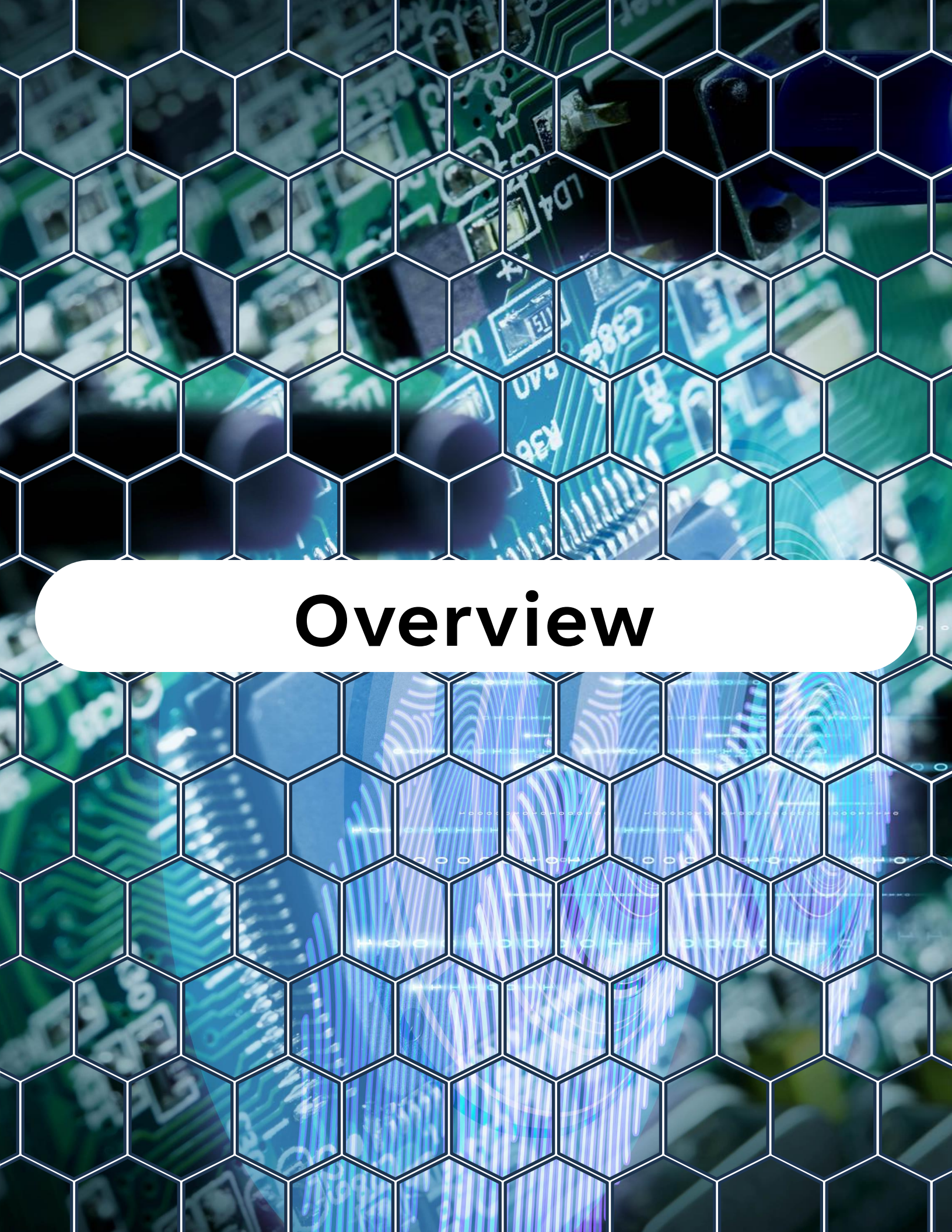


Table of Contents

Table of Contents

1. Overview
2. Common Debugging Challenges in FreeRTOS
3. Task Debugging Tools
4. Queue, Semaphore, and Event Group Debugging
5. Runtime Stats and Tracing
6. ESP-IDF Debugging Tools
7. Example – Debugging Tools in Action
8. Best Practices
9. Summary
10. Challenge for Today



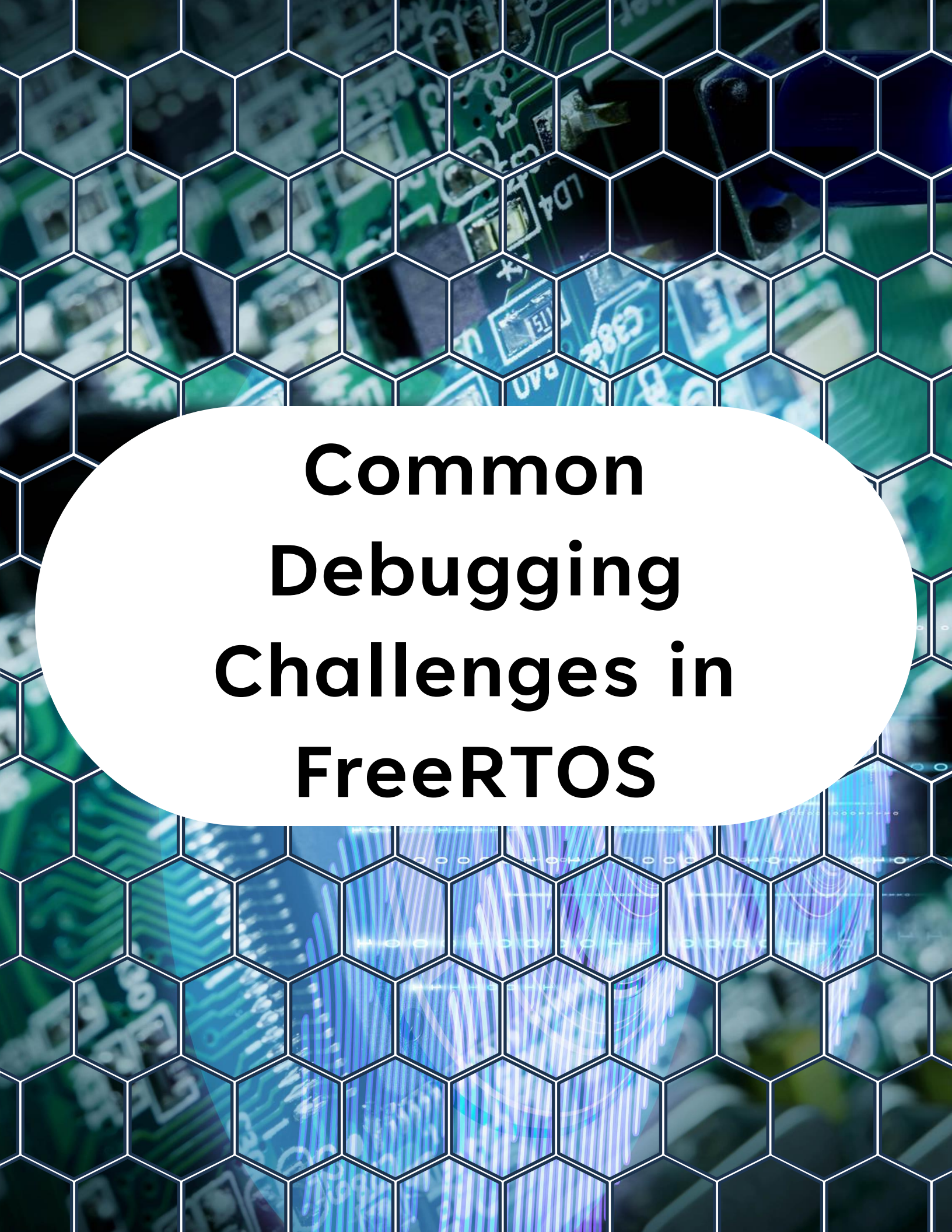
Overview

Overview

On **Day 23**, you'll learn:

- Common issues in FreeRTOS applications and how to debug them
- Built-in FreeRTOS **runtime stats and tracing tools**
- ESP-IDF debugging tools for **tasks, queues, and semaphores**
- Using **stack high-water marks and heap monitoring**
- Practical ESP32 debugging examples
- Best practices for diagnosing real-time system issues

By the end of this lesson, you'll know how to **track down and fix bugs** in FreeRTOS-based systems with confidence.



Common Debugging Challenges in FreeRTOS

Common Debugging Challenges in FreeRTOS

Typical issues include:

- **Task crashes** (stack overflow, illegal memory access)
- **Priority inversion** (Day 13)
- **Deadlocks** (tasks waiting forever on semaphores/queues)
- **Memory leaks or heap exhaustion**
- **Starvation** (low-priority tasks never get CPU time)



FreeRTOS and ESP-IDF provide several tools to help find these issues.



Task Debugging Tools

Task Debugging Tools

Stack High-Water Mark

Check if a task is close to overflowing its stack:



```
1 BaseType_t uxHighWaterMark;  
2 uxHighWaterMark = uxTaskGetStackHighWaterMark(task_handle);  
3 printf("Task high-water mark: %u words\n", uxHighWaterMark);
```

- Low values = stack usage is near the limit.
- Increase stack size if necessary.

Task List

Enable runtime stats in menuconfig:

Component config → FreeRTOS → Enable
FreeRTOS runtime stats

Then:



```
1 vTaskList(buffer);
```



Queue, Semaphore, and Event Group Debugging

Queue, Semaphore, and Event Group Debugging

Queue State



```
1 // Example code snippet to check the number
2 // of messages in a FreeRTOS queue
3 UBaseType_t count;
4 count = uxQueueMessagesWaiting(queue);
5 printf("Queue has %u messages\n", count);
```

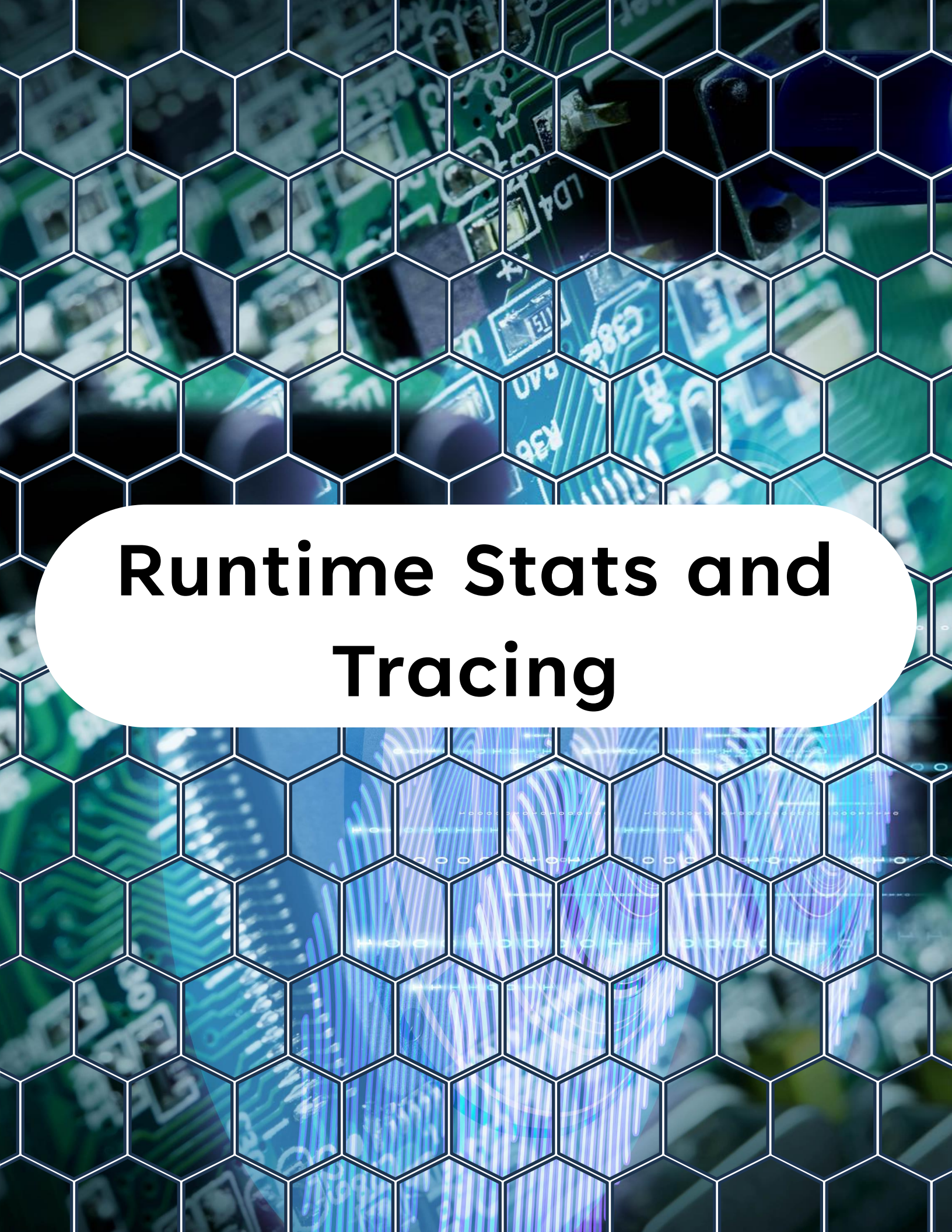
Semaphore Count



```
1 // Get the current count of the semaphore
2 UBaseType_t sem_count = uxSemaphoreGetCount(sem);
3 printf("Semaphore count: %u\n", sem_count);
```

Event Groups

Log event bits before/after waits to confirm synchronization.



Runtime Stats and Tracing

Runtime Stats and Tracing

CPU Usage (per task)

Enable `configGENERATE_RUN_TIME_STATS` in `FreeRTOSConfig.h`.

Then use:

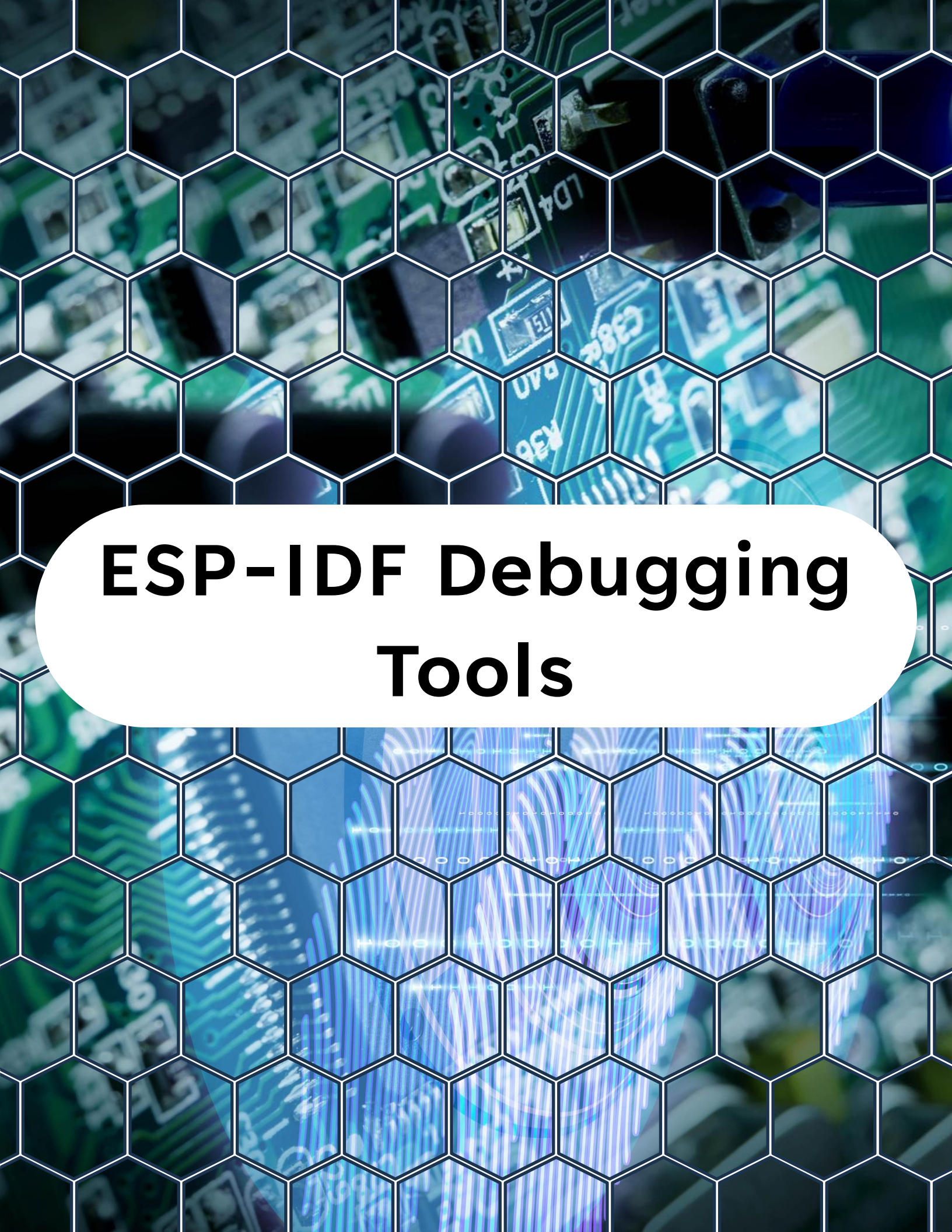


```
1 // Get runtime statistics
2 vTaskGetRunTimeStats(buffer);
```

Shows how much CPU each task consumes.

Trace Hooks

FreeRTOS has **trace macros** that let you hook into task switches, queue events, etc., for advanced profiling (often used with external tools like Tracealyzer).



ESP-IDF Debugging Tools

ESP-IDF Debugging Tools

ESP-IDF provides additional features for ESP32 debugging:

- **ESP-IDF Monitor**

- Run with `idf.py monitor` → captures crash logs and exceptions.
- Decodes backtraces into function names (requires `idf.py` add-dependency or symbol info).

- **Heap Debugging**

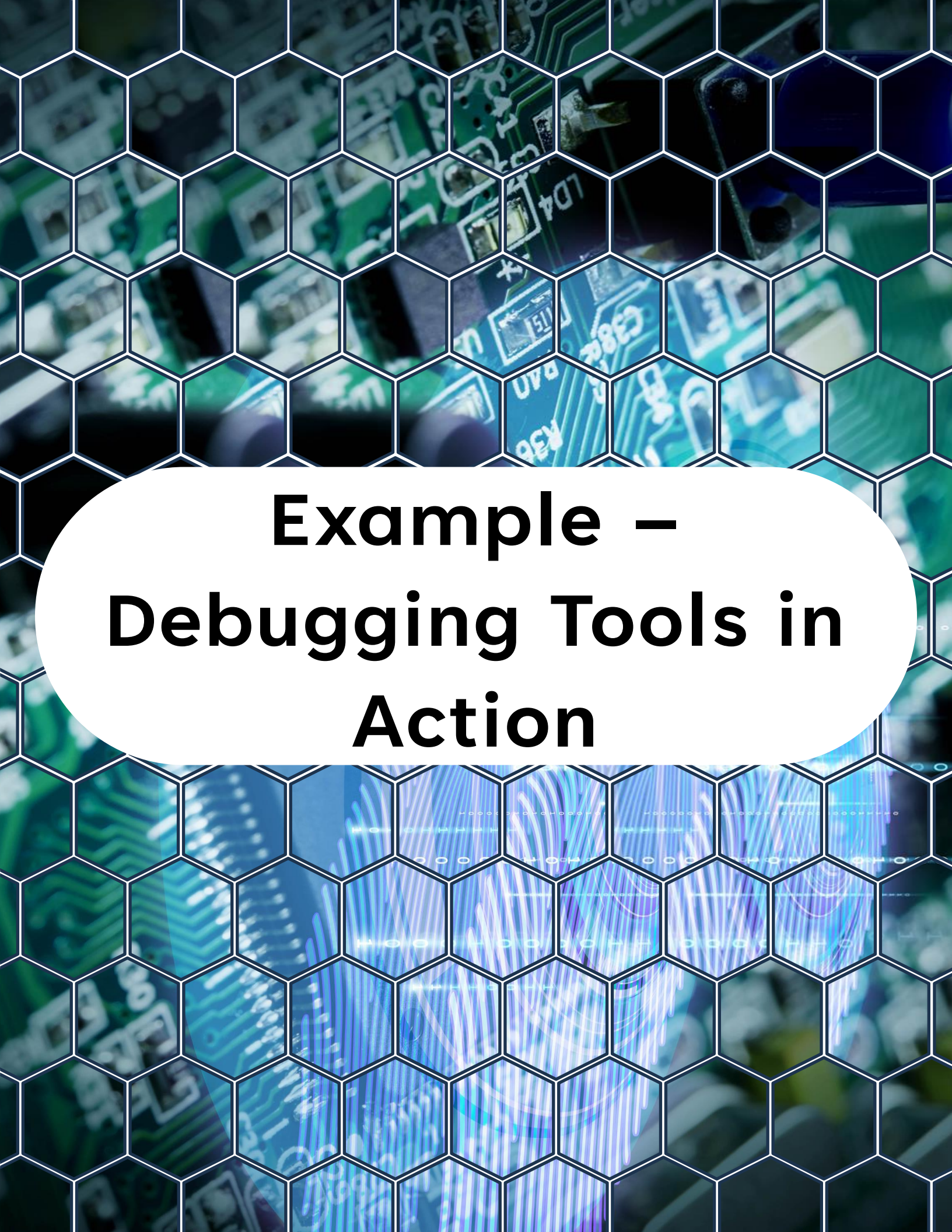


```
1 // Print out heap information
2 heap_caps_print_heap_info(MALLOC_CAP_8BIT);
```

- **Stack Overflow Checking**

Enable in `menuconfig`:

Component config → FreeRTOS → Check for stack overflow



Example – Debugging Tools in Action

Example – Debugging Tools in Action

Code Example

```
1  /**
2   * @file main.c
3   *
4   * @brief This code demonstrates the usage of FreeRTOS task
5   *        management functions to debug and monitor tasks.
6   *
7   * @note This project requires to enable the runtime stats
8   *        collection in FreeRTOSConfig.h:
9   *        KCONFIG Name: FREERTOS_GENERATE_RUN_TIME_STATS
10  */
11 #include <stdio.h>
12 #include "freertos/FreeRTOS.h"
13 #include "freertos/task.h"
14
15 // Example task to demonstrate usage of FreeRTOS task functions
16 void debug_task(void *pvParameters) {
17     while (1) {
18         // Print stack high-water mark
19         UBaseType_t watermark = uxTaskGetStackHighWaterMark(NULL);
20         printf("DebugTask high-water mark: %u words\n", watermark);
21
22         vTaskDelay(pdMS_TO_TICKS(1000));
23     }
24 }
25
26 // Main application entry point
27 void app_main() {
28     // Create the debug task
29     xTaskCreate(debug_task, "DebugTask", 2048, NULL, 5, NULL);
30
31     // Example: Print runtime stats periodically
```

Example – Debugging Tools in Action

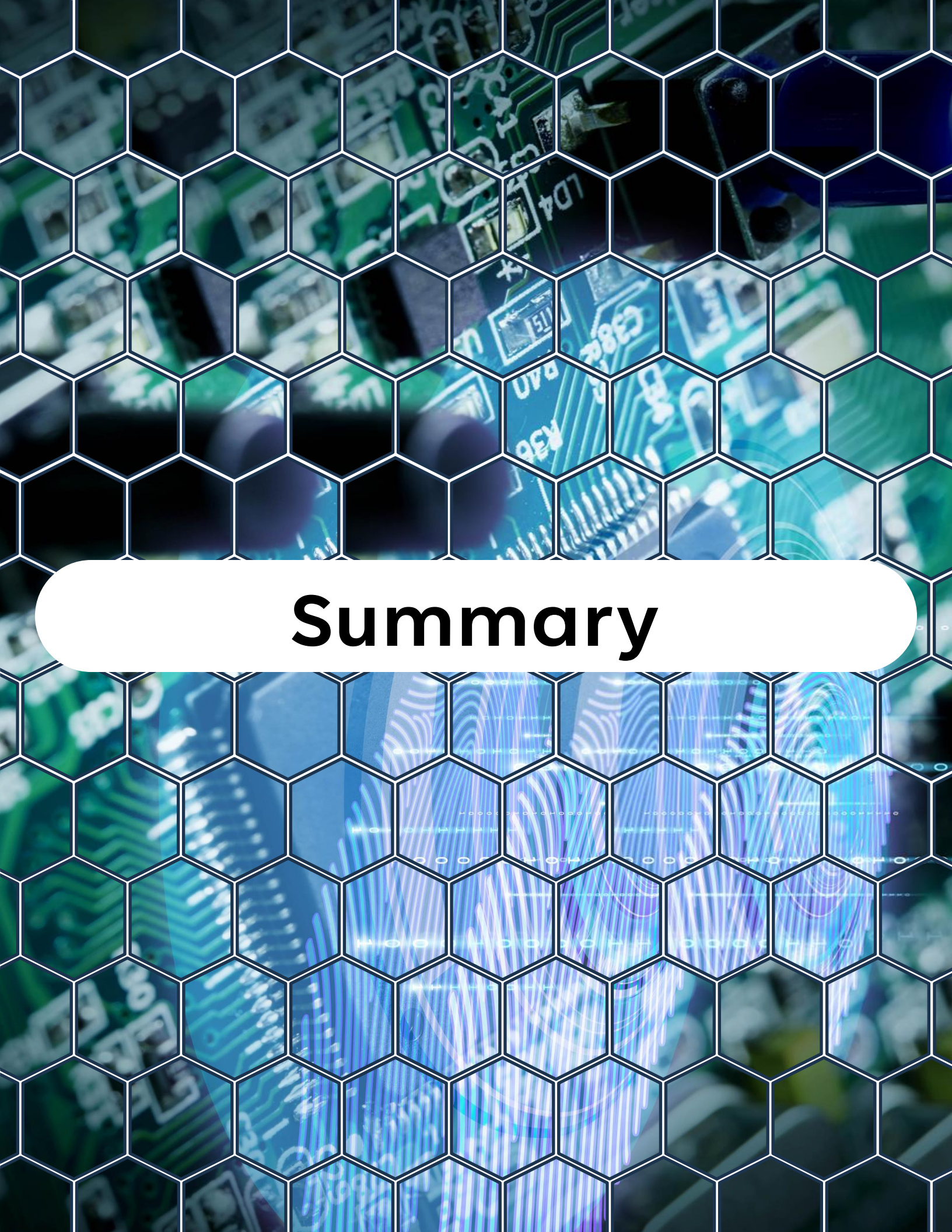
```
10 //
11 #include <stdio.h>
12 #include "freertos/FreeRTOS.h"
13 #include "freertos/task.h"
14
15 // Example task to demonstrate usage of FreeRTOS task functions
16 void debug_task(void *pvParameters) {
17     while (1) {
18         // Print stack high-water mark
19         UBaseType_t watermark = uxTaskGetStackHighWaterMark(NULL);
20         printf("DebugTask high-water mark: %u words\n", watermark);
21
22         vTaskDelay(pdMS_TO_TICKS(1000));
23     }
24 }
25
26 // Main application entry point
27 void app_main() {
28     // Create the debug task
29     xTaskCreate(debug_task, "DebugTask", 2048, NULL, 5, NULL);
30
31     // Example: Print runtime stats periodically
32     char buffer[512];
33     while (1) {
34         vTaskDelay(pdMS_TO_TICKS(5000));
35         vTaskList(buffer);
36         printf("Task List:\n%s\n", buffer);
37
38         vTaskGetRunTimeStats(buffer);
39         printf("Runtime Stats:\n%s\n", buffer);
40     }
41 }
```



Best Practices

Best Practices

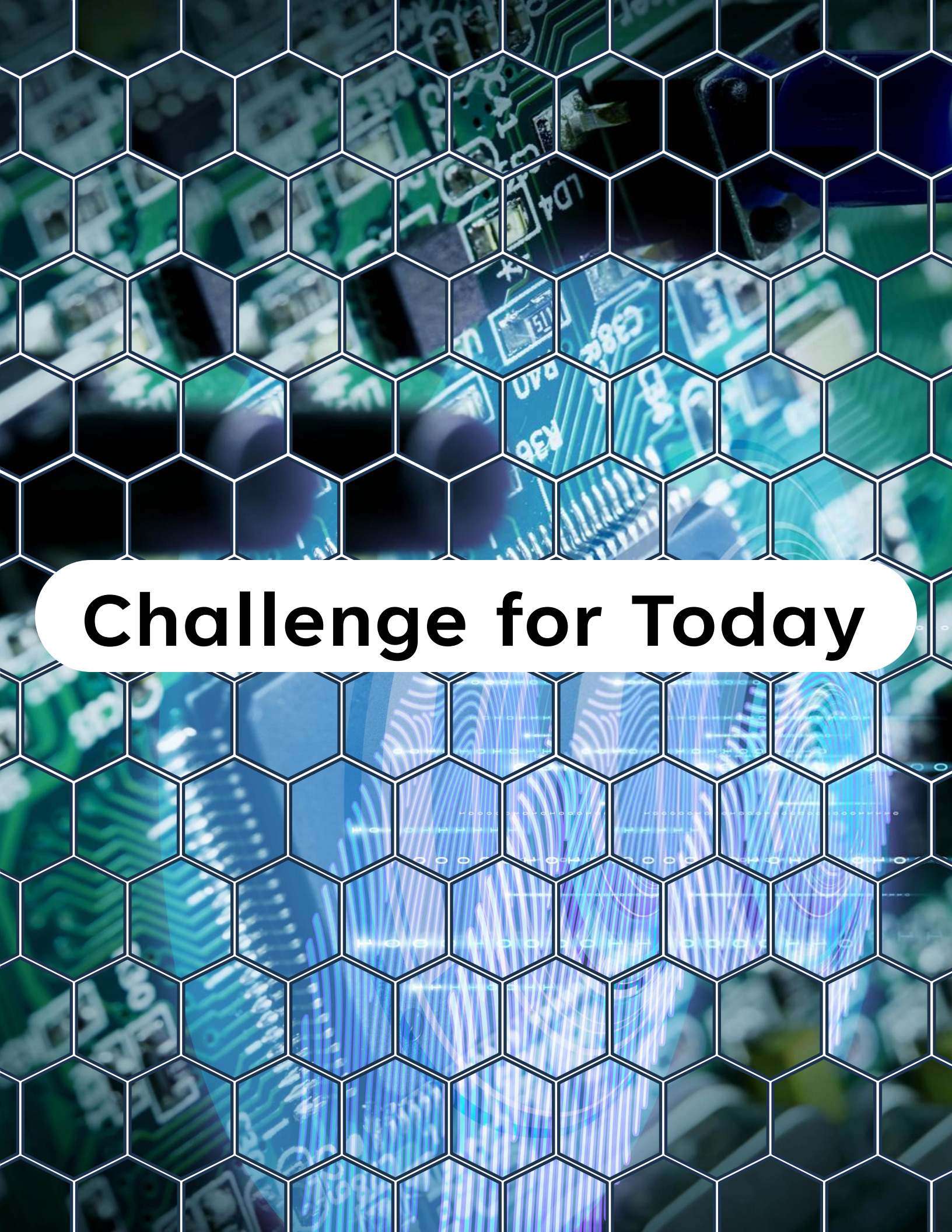
- ✓ Always enable stack overflow checking in development builds.
- ✓ Use `uxTaskGetStackHighWaterMark()` early to size task stacks properly.
- ✓ Monitor minimum heap size with `xPortGetMinimumEverFreeHeapSize()`.
- ✓ Use queues with timeouts (never block forever) to avoid deadlocks.
- ✓ For complex systems, use trace tools (Tracealyzer, SystemView) for visualization.



Summary

Summary

- Debugging in FreeRTOS requires monitoring **tasks, queues, semaphores, and memory.**
- Use stack high-water marks and runtime stats for insight.
- ESP-IDF provides tools like **monitor, heap info, and stack checks.**
- Always test under stress to catch timing and concurrency issues.



Challenge for Today

Challenge for Today



Extend today's example by:

1. Creating two tasks with **small stacks** and monitoring their high-water marks.
2. Intentionally trigger a **stack overflow** and observe ESP-IDF's crash report.
3. Use `vTaskGetRunTimeStats()` to profile which task consumes the most CPU.



"Thanks for watching!

*If you enjoyed this video, make sure to hit
the like button and subscribe to stay
updated with my latest content.*

*Don't forget to check out my other videos
for more tips and tutorials on Embedded
C, Python, hardware designs, etc. Keep
exploring, keep learning, and I'll see you in
the next video!"*

<https://www.linkedin.com/in/yamil-garcia>
<https://www.youtube.com/@LearningByTutorials>
<https://github.com/god233012yamil>